

gradient-descent-lean: Formal Proofs of Gradient Descent Convergence in Lean 4

Ben Cassie
2026

Abstract

gradient-descent-lean is a Lean 4 / Mathlib library formalising convergence results for gradient descent in smooth convex optimisation. The library contains 18 proved theorem statements with zero sorry, zero admit, and no project-specific axioms, over arbitrary real Hilbert spaces rather than a finite-dimensional Euclidean special case. Its main results are the standard $O(1/k)$ convergence rate for gradient descent on smooth convex objectives and a geometric convergence rate for smooth strongly convex objectives. The development builds on Mathlib's HasGradientAt, LipschitzWith, and real inner-product-space infrastructure. The significance is twofold: it gives a machine-checked reference for foundational optimisation theory, and it provides an importable Lean artifact for future formal work on machine learning, optimisation, and AI safety, where gradient descent is both a mathematical primitive and a practical engine of modern learning systems.

1. Introduction

Gradient descent is one of the central algorithms of modern optimisation. It appears in numerical analysis, convex optimisation, machine learning, reinforcement learning, control theory, and contemporary AI training. Many theoretical arguments in these areas reduce to a small set of standard facts: smoothness implies a quadratic upper bound, gradient steps decrease the objective, convexity turns local first-order information into global comparison inequalities, and strong convexity upgrades sublinear convergence into geometric contraction.

These facts are often presented as elementary textbook arguments. However, their apparent simplicity hides many details: differentiability hypotheses, conventions for gradients in inner-product spaces, constants such as $L/2$, step-size side conditions, telescoping estimates, and the distinction between function-value convergence and distance convergence. In informal mathematics these details are usually handled by convention. In a formal proof assistant, they must be stated and checked precisely.

gradient-descent-lean formalises this convergence spine in Lean 4 / Mathlib. The library proves the standard $O(1/k)$ rate for smooth convex optimisation and the geometric rate for smooth strongly convex optimisation. The results are stated over arbitrary real Hilbert spaces, using Mathlib's inner-product-space and gradient APIs, rather than specialising to $\mathbb{R}^n$.

The contribution is not a new optimisation theorem. It is a machine-checked, importable proof artifact for a foundational algorithm. This matters for AI safety and formal machine learning because gradient descent is a basic object in reasoning about training dynamics, loss landscapes, stability, and alignment procedures. If future

formal work is to reason about learning systems, it needs reliable formal foundations for the optimisation primitives those systems use.

At the time of writing, Mathlib contains substantial analysis, topology, normed-space, and calculus infrastructure, but no compact standalone Lean 4 library packaging the classical gradient descent convergence-rate proof in this form. `gradient-descent-lean` fills that gap.

2. Library Overview

The project is organised into four Lean modules:

Module	Role
<code>GradientDescent/Defs.lean</code>	Core definitions: smoothness, strong convexity, convexity, minimisers, and the gradient descent sequence
<code>GradientDescent/Smooth.lean</code>	Smoothness consequences: quadratic upper bound and descent lemma
<code>GradientDescent/Convex.lean</code>	Convex and strongly convex first-order inequalities
<code>GradientDescent/Convergence.lean</code>	One-step estimates, telescoping bounds, $O(1/k)$ convergence, and strongly convex geometric convergence

The project depends on:

- Lean v4.28.0
- Mathlib v4.28.0

The formal setting is an arbitrary type E equipped with:

```
[NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
```

Thus the proofs are not restricted to finite-dimensional coordinate spaces. The gradient API is based on Mathlib's `HasGradientAt`, and smoothness is represented via a Lipschitz condition on the gradient.

The repository is available at:

<https://github.com/velvetmonkey/gradient-descent-lean>

3. Theorem Inventory

The source audit found 18 theorem declarations. The gradient descent sequence `gdSeq` itself is a definition, not a theorem; the associated recurrence facts are `gdSeq_zero` and `gdSeq_succ`.

Layer 1 - Foundations

1. `isLSmooth_def` — Characterises L-smoothness as existence of gradients everywhere plus an L-Lipschitz gradient map.
2. `lsmooth_upper_bound` — Proves the standard quadratic upper bound for smooth functions.
3. `descent_lemma` — Shows that a gradient step decreases the function value under the usual step-size condition.
4. `stronglyConvex_def` — Unfolds the definition of strong convexity used by the library.
5. `isConvex_lower_bound` — Gives the first-order lower bound for convex functions.
6. `isConvex_minimiser_gradient_zero` — Shows that the gradient vanishes at a minimiser.
7. `stronglyConvex_func_gap_lower` — Lower-bounds the function gap by a quadratic distance term under strong convexity.
8. `stronglyConvex_inner_gradient_bound` — Gives the inner-product lower bound used in the strongly convex contraction proof.

Layer 2 - Gradient Descent Sequence and Convergence

9. `gdSeq_zero` — The gradient descent sequence starts at the initial point.
10. `gdSeq_succ` — The successor step is $x_{n+1} = x_n - \alpha \nabla f(x_n)$.
11. `norm_sq_gd_expand` — Algebraic expansion of the squared norm after subtracting a scaled direction.
12. `gd_step_distance_sq` — Distance-to-solution identity after one gradient step.
13. `gd_convex_step_bound` — One-step convex-case bound relating progress to distance decrease.
14. `gd_descent_nonincreasing` — Gradient descent produces non-increasing objective values.
15. `gd_convex_telescope` — Telescoping estimate for the convex convergence proof.
16. `gd_convex_convergence` — Proves the $O(1/k)$ convex convergence rate.
17. `gd_strongly_convex_step` — Proves the one-step contraction estimate in the strongly convex case.
18. `gd_strongly_convex_convergence` — Proves the geometric strongly convex convergence rate.

The two headline convergence results are:

- Smooth convex case: function-value error decays at rate $O(1/k)$.
- Smooth strongly convex case: squared distance to the minimiser contracts geometrically.

4. Key Technical Highlights

Smoothness Upper Bound

A central technical result is `lsmooth_upper_bound`, the standard inequality

$$f(y) \leq f(x) + \langle \text{grad } f \ x, y - x \rangle + (L / 2) * ||y - x||^2.$$

In many optimisation texts this result is introduced as a lemma and then reused throughout convergence proofs. In `gradient-descent-lean`, it is proved from the smoothness definition rather than assumed. The proof uses Mathlib’s calculus and integration infrastructure, following the usual fundamental-theorem-of-calculus argument along the line segment from x to y .

This matters because the quadratic upper bound is the bridge between analytic smoothness and algorithmic descent. Once formalised, later descent and convergence arguments can depend on a checked lemma rather than an informal imported fact.

Hilbert Space Generality

The library works over arbitrary real Hilbert spaces:

```
[NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]
```

This is stronger than a proof specialised to finite-dimensional Euclidean vectors. It means the convergence theorems are expressed at the level of Mathlib’s abstract normed and inner-product-space hierarchy. The same theorem statements can therefore support later work involving function spaces, infinite-dimensional optimisation, or abstract training dynamics, provided the relevant hypotheses are available.

Standard Axioms Only

The development introduces no new axioms. The proof audit reports only the standard Lean/Mathlib axioms:

- `propext`
- `Classical.choice`
- `Quot.sound`

This is the ordinary trusted base expected for noncomputable classical mathematics in Lean. The project contains `zero sorry` and `zero admit`.

Importable Optimisation Spine

The library is intentionally small. Its main value is as a reusable proof spine: future projects can import the definitions and convergence facts rather than re-formalising the smooth convex gradient descent argument from scratch.

5. Relation to `kuramoto-lean`

This project is part of the same AI-assisted Lean 4 formalisation workflow as `kuramoto-lean`, also by Ben Cassie.

The two libraries are complementary:

- `kuramoto-lean` formalises finite- N Kuramoto synchronisation dynamics, gradient identities, Lyapunov descent, Hebbian coupling, and frontier dynamical-systems results.

- `gradient-descent-lean` formalises optimisation convergence for gradient descent, including the $O(1/k)$ convex rate and the geometric strongly convex rate.

Both projects use Lean 4 and Mathlib to machine-check mathematical claims that are foundational to modern AI reasoning. `kuramoto-lean` focuses on dynamical systems and synchronisation. `gradient-descent-lean` focuses on optimisation theory. Together they support a broader programme of formalising learning and observer dynamics: gradient flows, Lyapunov descent, and convergence mechanisms as checked mathematical artifacts.

Related links:

- `gradient-descent-lean`: <https://github.com/velvetmonkey/gradient-descent-lean>
- `kuramoto-lean`: <https://github.com/velvetmonkey/kuramoto-lean>
- `kuramoto-lean` Zenodo DOI: <https://doi.org/10.5281/zenodo.20469680>

6. Significance for AI Safety

Gradient descent is not merely a numerical method; it is the mechanism underlying most modern machine learning. Claims about stability, convergence, loss landscapes, training dynamics, and alignment interventions often rely on informal optimisation arguments. Formalising those arguments makes their assumptions explicit.

A machine-checked convergence theorem cannot certify a deployed AI system by itself. However, it can provide reliable mathematical components for larger formal arguments. For example, one may want to reason about whether a training update decreases a surrogate loss, whether a regularised objective induces contraction, or whether a simplified model of learning converges under stated hypotheses. Libraries such as `gradient-descent-lean` provide the reusable primitives needed for that work.

The value is cumulative. A small verified theorem about gradient descent becomes useful when imported into a larger proof about optimisation, control, reinforcement learning, or alignment. The purpose of this library is to make one foundational part of that stack precise and reusable.

7. Conclusion

`gradient-descent-lean` provides a compact Lean 4 / Mathlib formalisation of classical gradient descent convergence theory. It proves 18 theorem statements with zero sorry, over arbitrary real Hilbert spaces, including the standard $O(1/k)$ smooth convex rate and the geometric smooth strongly convex rate.

The project is deliberately focused. It does not attempt to formalise stochastic gradient descent, nonconvex optimisation, adaptive methods, or deep-learning-specific training dynamics. Instead, it supplies a reliable formal core for deterministic smooth convex gradient descent. That core can now be imported, extended, and used as a foundation for more ambitious formal work in optimisation, machine learning, and AI safety.

References

Nesterov, Y. (2004). *Introductory Lectures on Convex Optimization: A Basic Course*. Springer.

The Mathlib Community. (2024). *The Lean Mathematical Library*. GitHub repository. <https://github.com/leanprover-community/mathlib4>

Cassie, B. (2026). *kuramoto-lean: A Sorry-Free Lean 4 Library for Finite-N Kuramoto Synchronisation Dynamics*. Zenodo. <https://doi.org/10.5281/zenodo.20469680>

Cassie, B. (2026). *gradient-descent-lean: Formal Proofs of Gradient Descent Convergence in Lean 4*. GitHub repository. <https://github.com/velvetmonkey/gradient-descent-lean>